



**INNOPOLIS
UNIVERSITY**

Введение в компьютерное зрение

Илья Афанасьев, к.т.н., ЛИРС

Лекция 2 · **Формирование изображения: оптика и сенсор**

Dr. Ilya Afanasyev, i.afanasyev@innopolis.ru

«Introduction to Computer Vision» course, Innopolis University (IU), Russia, Fall 2026

Блок I Изображение и его источник	Блок II Операции над изображением	Блок III Признаки	Блок IV Гомография и геометрия	Блок V Видео и движение
L1 Введение. Изображение и видео как данные	L4 Геометрические преобразования	L7 Границы, линии и робастное оценивание	L10 Гомография: оценивание и выравнивание	L13 Видео: движение и оптический поток
L2 Формирование изображения: оптика и сенсор	L5 Пространственная фильтрация	L8 Точки интереса: детекторы	L11 Модель камеры и калибровка	L14 Трекинг и распознавание объектов
L3 Цвет, гистограммы и точечные операции	L6 Частотная область и пирамиды	L9 Дескрипторы и сопоставление	L12 Стерео и эппиплярная геометрия	L15 Синтез: от двух видов к 3D. Итоги

Сегодня — L2: Формирование изображения: оптика и сенсор

RECAP

- Зрение — **обратная задача**: в кадре уже нет ни глубины, ни спектра, ни времени
- Пиксель — интеграл спектра $\int L(\lambda) \cdot S(\lambda) d\lambda$ и **угол** FOV / W, а не «цвет» и не длина
- Код 128 — не половина света: в uint8 спрятана гамма и сегодня увидим, как **ISP** её даёт
- Пересвет и смаз **необратимы** — увидим, где они рождаются: на сенсоре и на затворе

Разбор самопроверки L1 — прямо сейчас

кадр 4K — это ≈ 25 МБ, маску ресемплим только NEAREST, срез даёт shape (100, 10, 3) и меняет оригинал, а два спектра с одинаковым RGB — метамерия.

LEARNING OBJECTIVES

- **Вычислить** проекцию точки: $u = f_x \cdot X/Z + c_x$ — и получить f_x из поля зрения
- **Объяснить** цену каждой ручки экспозиции: смаз, глубина резкости, шум
- **Проследить** путь фотона до JPEG и назвать, что теряет каждая стадия ISP
- **Измерить** шум сенсора по стеку кадров и предсказать его законом $\sqrt{N}$
- **Распознать** артефакты демозаика и rolling shutter в чужом кадре

Задача: метры из пикселей

МОТИВАЦИЯ

Дано: кадр Full HD и пешеход ростом 141 px. Найти: дистанцию в метрах

- В лоб никак: у пикселя нет длины — 141 px это и 2 м, и 200 м
- С моделью: $Z = f \cdot h / H = 1663 \cdot 1.7 / 141 = \mathbf{20 \text{ м}}$
- Робототехника: монокулярная дальность; ночью тот же кадр тонет в шуме
- МОИИ: аугментации имитируют оптику и шум — надо знать, что имитировать

Чтобы решить задачу, нужна модель камеры и калибровка

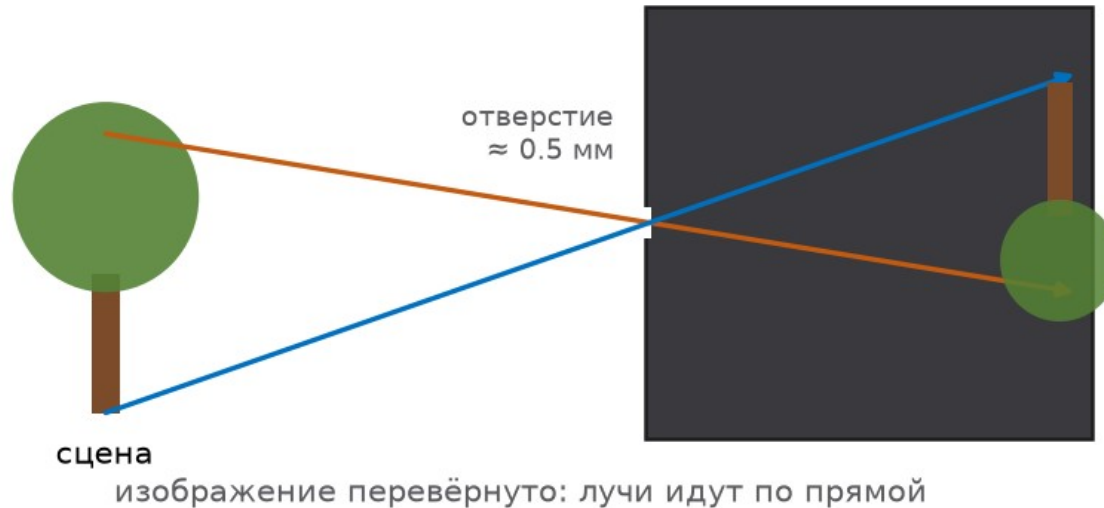
1

Оптика: от сцены к проекции

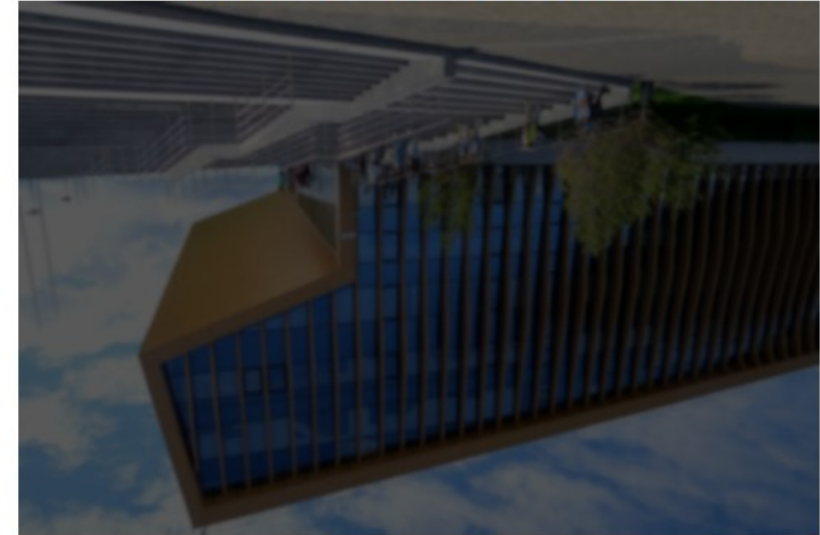
- Pinhole-модель и её пределы; линза, фокус, диафрагма, дисторсия

Камера-обскура: первая камера

Камера-обскура: тёмная комната, отверстие, задняя стенка



что на задней стенке: перевёрнуто и темно



Тысячу лет назад Ибн ал-Хайсам описал: свет в тёмную комнату проходит через отверстие, и на стенке — перевёрнутая сцена. В эпоху Ренессанса художники соединили геометрию прямых лучей света с искусством. Они заставили плоскую картину подчиняться тем же законам физики и оптики, по которым устроен человеческий глаз, и совершили революцию в искусстве рисования. Лучи идут по прямой — вся проекция следует из этого факта

© фото — здание Университета Иннополис; собственная иллюстрация

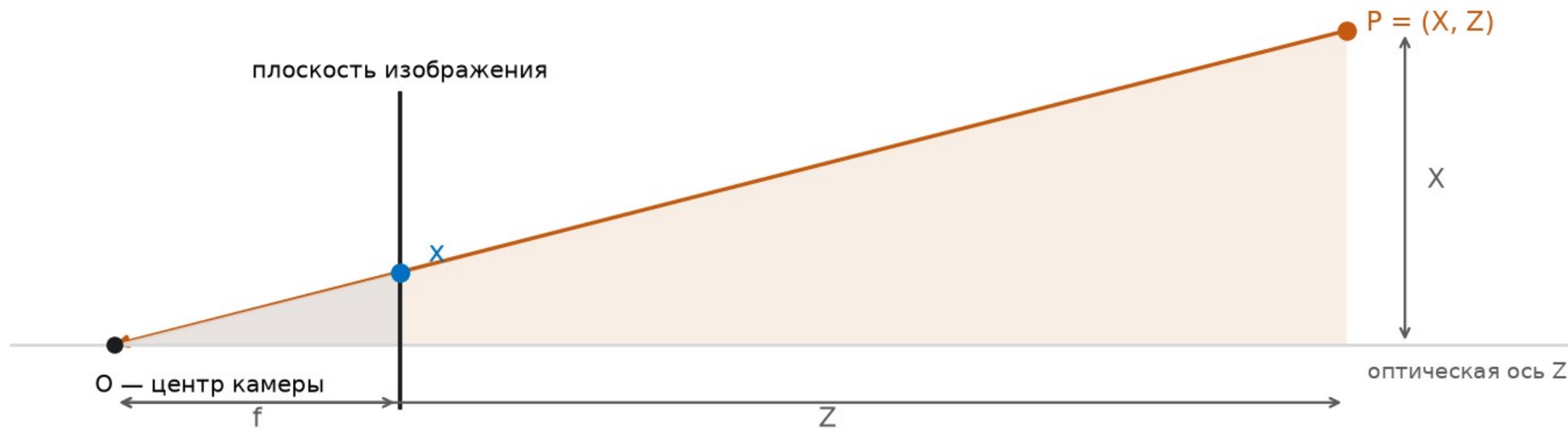
Формально

- Дано: точка сцены $P = (X, Y, Z)$ — в метрах, в системе координат камеры
- Найти: пиксель (u, v) , в который она проецируется
- Критерий: ошибка проекции в пикселях против реального кадра

Модель камеры = функция $(X, Y, Z) \rightarrow (u, v)$; сегодня строим её явно

Pinhole: проекция как подобие

подобные треугольники: $x / f = X / Z \rightarrow x = f \cdot X / Z$

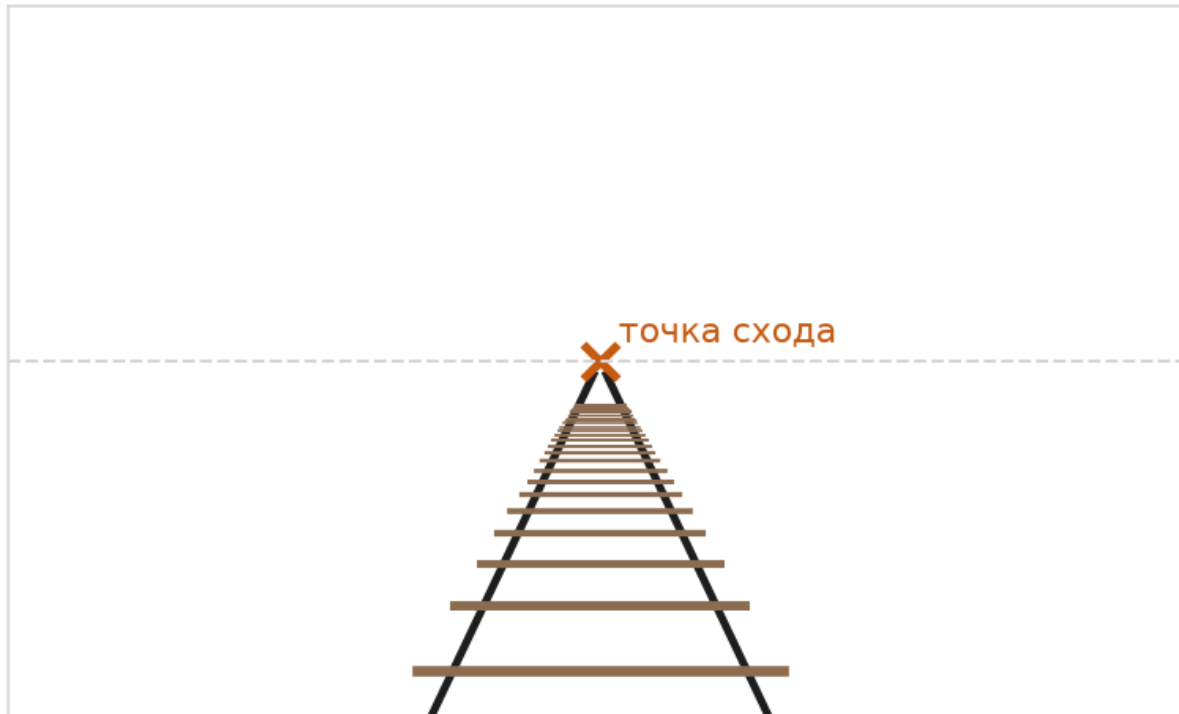


Луч из P проходит через центр O; подобные треугольники дают $x = f \cdot X / Z$. Это вся математика проекции: одна формула на весь курс

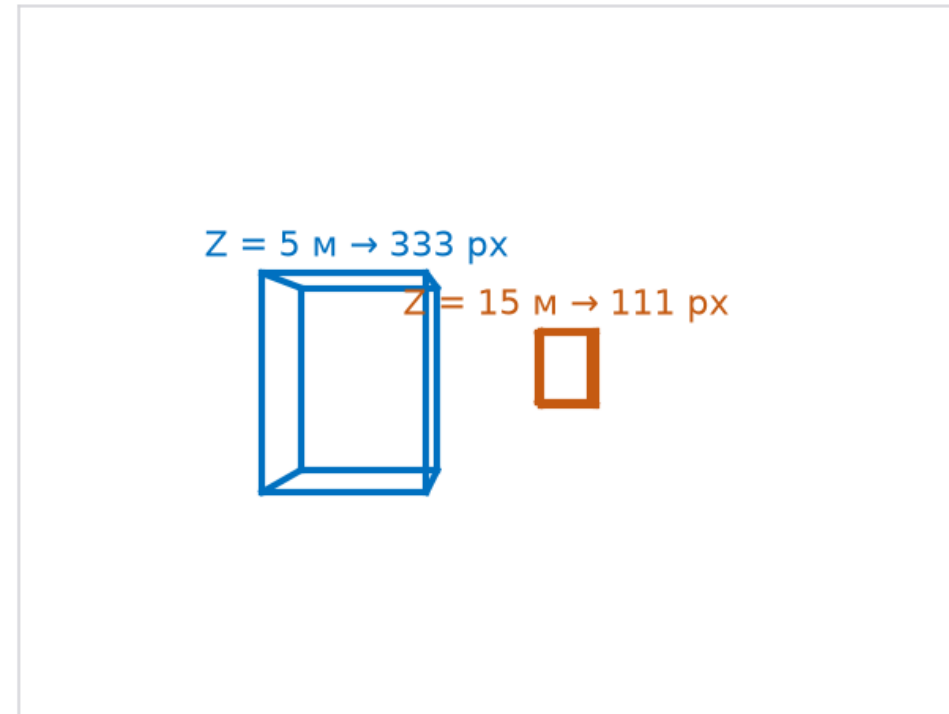
© собственная иллюстрация

Перспектива: всё делится на Z

параллельные рельсы (колея 1.52 м) сходятся в точку на горизонте



один и тот же куб 1 м: размер в кадре $\propto 1 / Z$



Деление на Z — это и есть перспектива: рельсы (колея 1.52 м) сходятся в точку, куб 1 м на 5 м — 333 px, на 15 м — 111 px. Прямые остаются прямыми, а длины и углы — нет

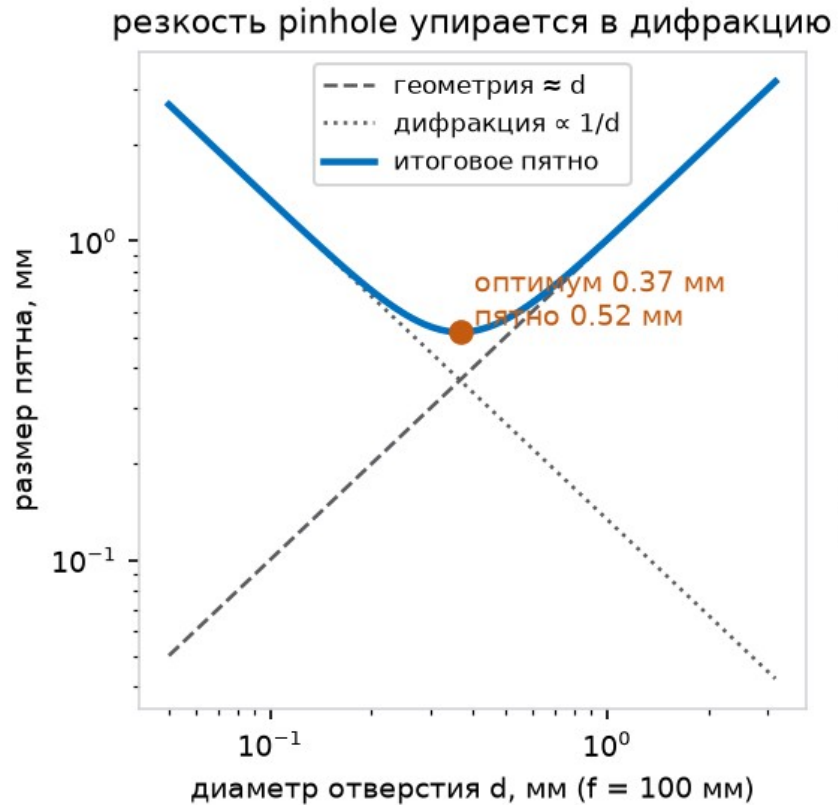
© собственная иллюстрация

- $u = f_x \cdot X/Z + c_x, v = f_y \cdot Y/Z + c_y$ — физически: масштаб и сдвиг начала координат
- f_x — фокусное расстояние **в пикселях**: $f_x = W / (2 \cdot \tan(FOV/2))$
- (c_x, c_y) — **главная точка** (principal point): куда падает оптическая ось, $\approx$ центр кадра
- f_x, f_y, c_x, c_y — интринсики; в L11 соберём их в матрицу K и измерим

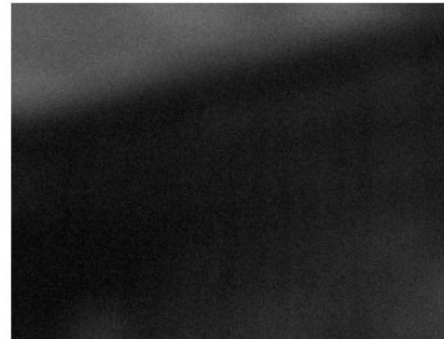
- Камера L1: FOV 60°, 1920 px $\rightarrow f_x = 960 / \tan 30^\circ = \mathbf{1663 \text{ px}}$
- Пешеход 1.7 м: на 10 м — **283 px**, на 20 м — **141 px**, на 42 м — 67 px
- Обратная задача из мотивации: $Z = f_x \cdot H / h = 1663 \cdot 1.7 / 141 = \mathbf{20 \text{ м}}$

Числа проверим кодом в демо 1.1–1.2

Почему pinhole непрактичен



0.1 мм: дифракция,
мало света



0.37 мм: оптимум —
всё равно мягко

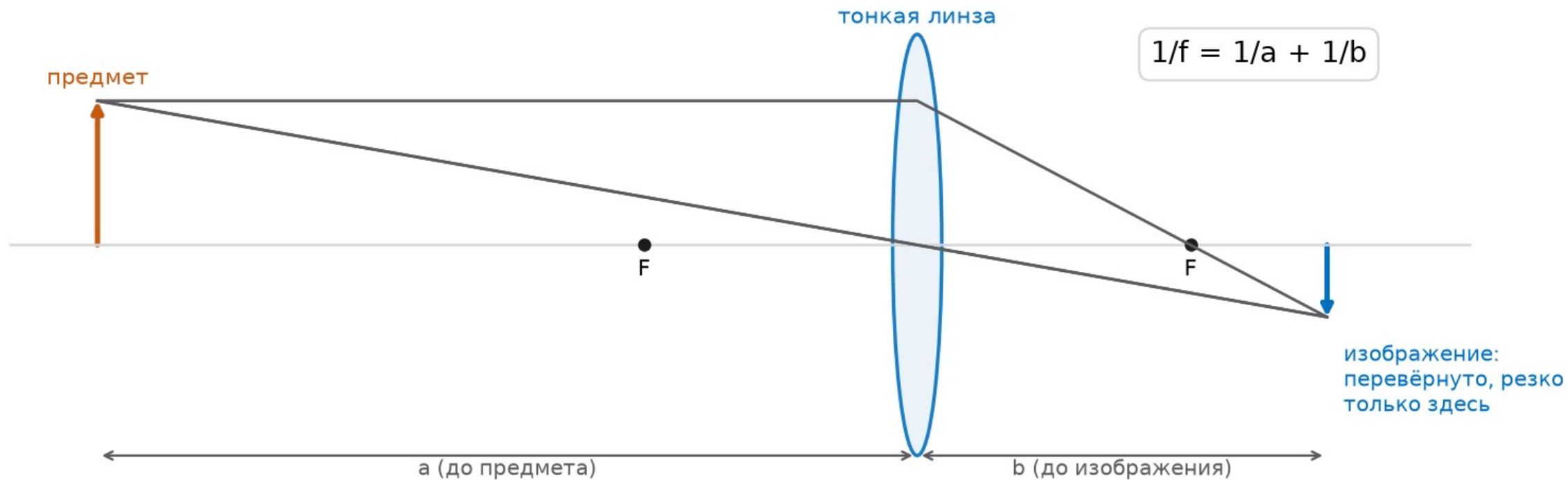


1.5 мм:
геометрическое мыло



Отверстие не может быть ни большим (геометрическое пятно $\approx d$), ни крошечным (дифракция $\propto 1/d$): лучший компромисс при $f = 100$ мм — $d \approx 0.37$ мм, пятно 0.52 мм — всё равно мягко, а света как у $f/273$

© фото — здание Университета Иннополис; собственная иллюстрация

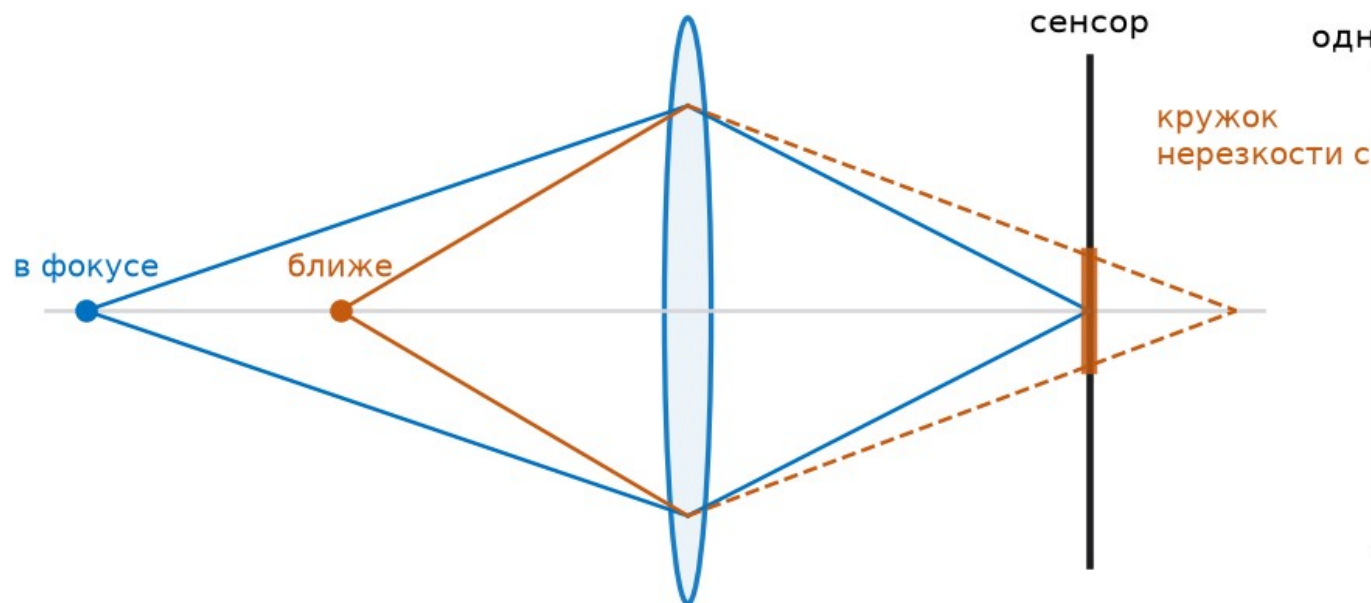


Линза собирает весь пучок из точки предмета в одну точку изображения: света в тысячи раз больше, чем у pinhole.
Плата — уравнение $1/f = 1/a + 1/b$: резко только на одной дистанции

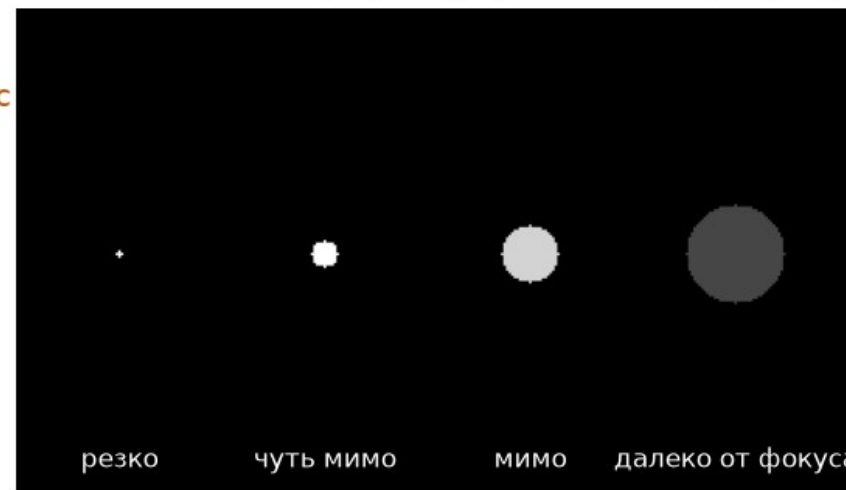
© собственная иллюстрация

Фокус и кружок нерезкости

не в фокусе — точка расплывается в кружок; с растёт с диафрагмой D



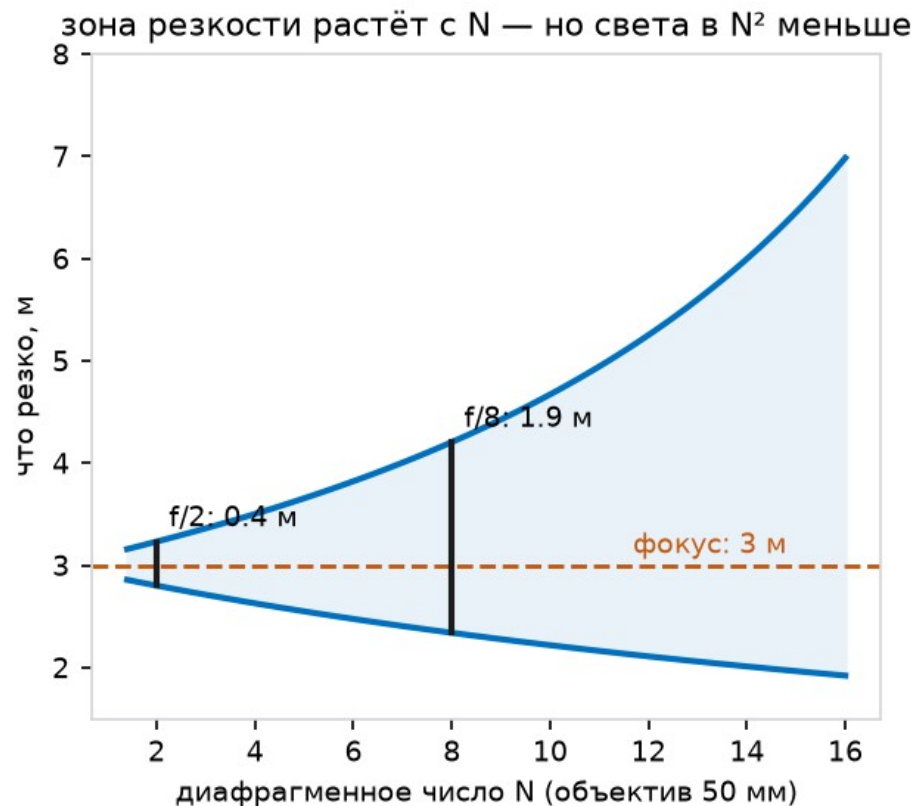
одна и та же точка при растущей ошибке фокусировки



Точка не в фокусе рисует на сенсоре кружок нерезкости (circle of confusion): пучок сошёлся до или после плоскости сенсора. Диаметр кружка растёт с ошибкой дистанции и с шириной пучка — то есть с диафрагмой

© собственная иллюстрация

- Диафрагма — дырка **внутри** объектива; характеристика — число $N = f / D$
- Света проходит $\propto D^2 \propto 1/N^2$: шаг стандартного ряда $f/1.4 \rightarrow f/2 \rightarrow f/2.8$ — **минус половина света**
- Прикрыли диафрагму $\rightarrow$ пучок узкий $\rightarrow$ кружок нерезкости меньше $\rightarrow$ **резкая зона глубже**
- Обозначение $f/2$ читается буквально: диаметр $D = f/2$, «дырка в половину фокусного»



$f/2$: фокус на 3 м — ГРИП 0.4 м



$f/8$: фокус на 3 м — ГРИП 1.9 м



Глубина резкости (depth of field) для 50 мм с фокусом на 3 м: при $f/2$ резко 2.80–3.23 м (0.43 м), при $f/8$ — 2.34–4.21 м (1.86 м). Справа сцена: при $f/2$ таблички «2 м» и «5 м» уже мыло

© фото — здание Университета Иннополис; собственная иллюстрация



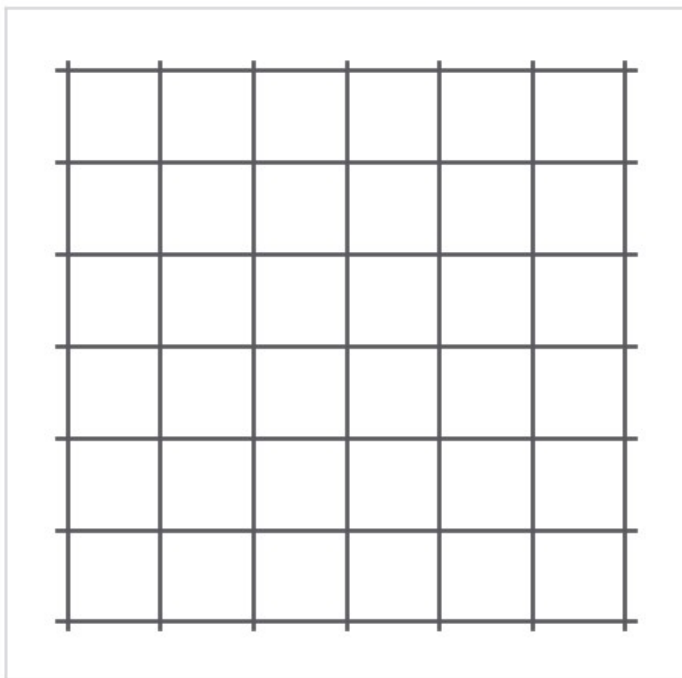
Выдержка, диафрагма и усиление добирают один и тот же свет, и у каждой ручки своя цена: смаз движения, малая глубина резкости, шум. Бесплатного света не бывает

© собственная иллюстрация

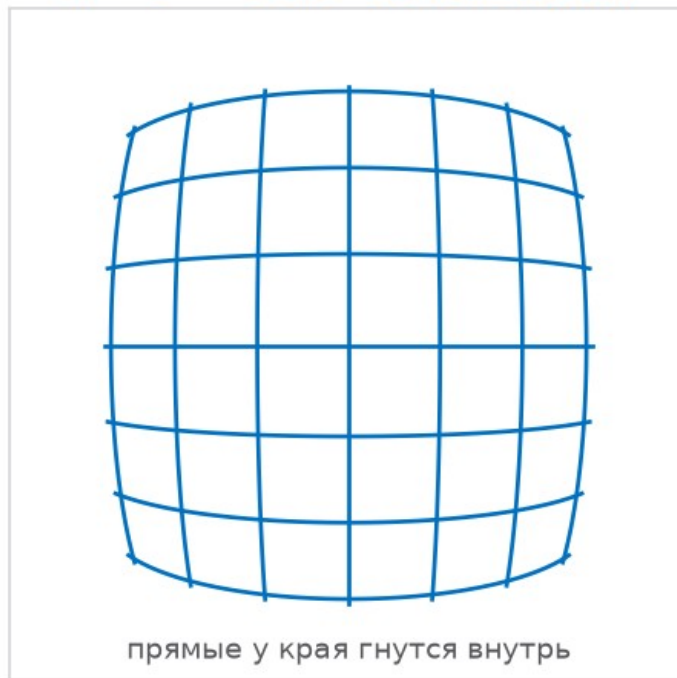
- Реальный объектив — не тонкая линза, а система линз; проекция **искривляется** к краям
- Радиальная модель: $r' = r \cdot (1 + k_1 r^2 + k_2 r^4)$ — **что это физически**: сдвиг точки вдоль радиуса от центра
- Знак k_1 задаёт тип: «бочка» (широкий угол) или «подушка» (телевик); есть ещё малая тангенциальная — перекося линз
- Модель — сегодня, **оценивание коэффициентов** — L11

радиальная дисторсия: $r' = r \cdot (1 + k_1 r^2 + k_2 r^4)$; у камеры курса $k_1 = -0.05$ сдвинул бы угол кадра на 23 px

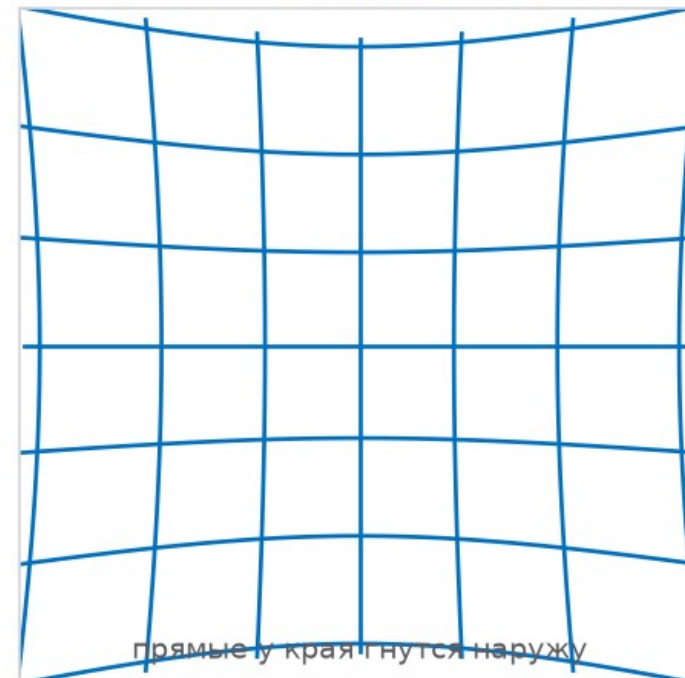
без дисторсии



$k_1 < 0$ — «бочка» (широкий угол)



$k_1 > 0$ — «подушка» (телевик)



У камеры курса $k_1 = -0.05$ сдвинул бы угол кадра на 23 px: прямая стена перестаёт быть прямой, и любое измерение без поправки врёт на десятки пикселей

© собственная иллюстрация

Параметры оптики и их смысл

Параметр	За что отвечает	Если увеличить
f (и f_x)	масштаб: px на угол	объект крупнее, поле уже
диафрагма N	свет и ширина пучка	темнее в N^2 , ГРИП глубже
выдержка t	сколько фотонов собрано	светлее, но смаз движения
фокусировка a	какой слой сцены резок	резкая зона дальше
k_1, k_2	кривизна проекции у краёв	сильнее гнёт прямые

- Проекция — одна формула $u = f_x \cdot X/Z + c_x$; перспектива — это деление на Z
- Линза покупает свет ценой фокусировки; диафрагма делит свет и ГРИП; дисторсия гнёт края
- Чего модель пока не объясняет: откуда в тёмном кадре **шум** — это вопрос к сенсору

Демо 1.1 — проекция куба

python

```
1 import numpy as np
2 fx, cx, cy = 1663.0, 960.0, 540.0    # камера L1: 60°, 1920 px
3 cube = np.array([[x, y, z] for x in (-.5, .5)
4                   for y in (-.5, .5) for z in (0, 1.0)])
5 for Z in (5.0, 15.0):
6     p = cube + [0, 0, Z]
7     u = fx * p[:, 0] / p[:, 2] + cx
8     v = fx * p[:, 1] / p[:, 2] + cy
9     print("куб 1 м на Z = %2.0f м: ширина %3.0f px"
10          % (Z, u.max() - u.min()))
```

куб 1 м на Z = 5 м: ширина 333 px · на Z = 15 м: ширина 111 px

Вызов

`fx * p[:, 0] / p[:, 2] + cx`

`u.max() - u.min()`

Что делает

вся модель камеры одной строкой: масштаб, деление на глубину, сдвиг к центру

ширина проекции куба в пикселях — по всем восьми углам сразу

Демо 1.2 — fx из поля зрения

```
1 W, fov = 1920, np.deg2rad(60.0)
2 fx = W / (2 * np.tan(fov / 2))
3 print("fx = %.0f px (%.2f мрад на пиксель)"
4       % (fx, 1000 * fov / W))
5 for Z in (10, 20, 42):
6     print("пешеход 1.7 м на %2d м: %3.0f px" % (Z, fx * 1.7 / Z))
```

python

fx = 1663 px (0.55 мрад на пиксель) ·
пешеход: 283 px (10 м), 141 px (20 м), 67 px (42 м)

Вызов

$W / (2 * \text{np.tan}(\text{fov} / 2))$

$\text{fx} * 1.7 / Z$

Что делает

fx из паспортного поля зрения: половина ширины кадра на тангенс половины угла

высота объекта 1.7 м в пикселях на дистанции Z — прямая формула проекции

Демо 1.3 — дисторсия в пикселях

python

```
1 k1 = -0.05 # лёгкая «бочка», как у веб-камеры
2 pts = np.array([[960.0, 540.0], [1910.0, 1070.0]])
3 n = (pts - [cx, cy]) / fx # в нормированные координаты
4 r2 = (n ** 2).sum(1, keepdims=True)
5 shift = np.linalg.norm(n * k1 * r2 * fx, axis=1)
6 print("сдвиг: центр %.1f px, угол кадра %.1f px" % (*shift,))
```

сдвиг: центр 0.0 px, угол кадра 23.3 px

Вызов

$(pts - [cx, cy]) / fx$

$n * k1 * r2 * fx$

Что делает

из пикселей в нормированные координаты: там модель дисторсии и живёт

радиальный сдвиг $k_1 \cdot r^2$ каждой точки, переведённый обратно в пиксели

Демо 1.4 — калькулятор ГРИП

python

```
1 f, c, s = 50e-3, 30e-6, 3.0    # 50 мм, допуск 30 мкм, фокус 3 м
2 for N in (2, 8):
3     H = f * f / (N * c) + f    # гиперфокальное расстояние
4     near = H * s / (H + s - f)
5     far = H * s / (H - s)
6     print("f/%d: резко %.2f...%.2f м (ГРИП %.2f м)"
7           % (N, near, far, far - near))
```

f/2: резко 2.80...3.23 м (ГРИП 0.43 м) · f/8:
резко 2.34...4.21 м (ГРИП 1.86 м)

Вызов

Что делает

$f * f / (N * c) + f$

гиперфокальное расстояние H: сфокусируйся туда — резко от H/2 до бесконечности

$H * s / (H \pm s - \dots)$

ближняя и дальняя границы резкости для фокуса на дистанции s

- **Вопрос 1.** Телефон «108 Мп» с пикселем 0.7 мкм против камеры 27 Мп с пикселем 1.4 мкм, сенсоры одного размера. Чей ночной кадр чище — и почему?
- **Вопрос 2.** Ночное фото: ISO 100 и 30 с — чистое; ISO 6400 и 1/2 с — шумное. Значит ли это, что «ISO добавляет шум»?

Ответы поднятием рук; разбор — во второй части, измерением

B1: 108 Мп (0.7 мкм) против 27 Мп (1.4 мкм). При одинаковом размере матрицы кадр с 27 Мп камеры будет чище. Даже если 108 Мп телефон объединит пиксели (binning) до тех же 27 Мп, он проиграет по следующим причинам: (1) Меньше полезная площадь. Межпиксельные стенки и проводка съедают много места. У мелких пикселей рабочая область меньше. (2) Высокий шум считывания. Процессор считывает сигнал со 108 миллионов точек. Каждое считывание добавляет собственный электронный шум. (3) Электрические наводки. Пиксели расположены слишком близко. Заряды «перетекают» друг в друга, создавая паразитные искажения.

B2: Зависимость от ISO. Нет, ISO не добавляет шум. Высокое ISO лишь делает видимым тот шум, который уже есть в сигнале. ISO — это просто усилитель. При ISO 6400 камера усилила слабый сигнал вместе с дефицитом света.

2

Сенсор и ISP: от фотонов к JPEG

- Фотодиод, затворы, шум, мозаика Байера и конвейер обработки

Пиксель — колодец для фотонов

фотоны: 1000 за выдержку



полная ёмкость (full well):
например, 6000 e^- — выше
только клиппинг (255)

600 e^- (QE $\approx$ 60 %)

фотодиод-«колодец»

усилитель + АЦП

+ шум чтения $\approx$ 2-3 e^-
+ тёмновой ток (растёт с t°)

DN = 512

число в RAW (12-14 бит)

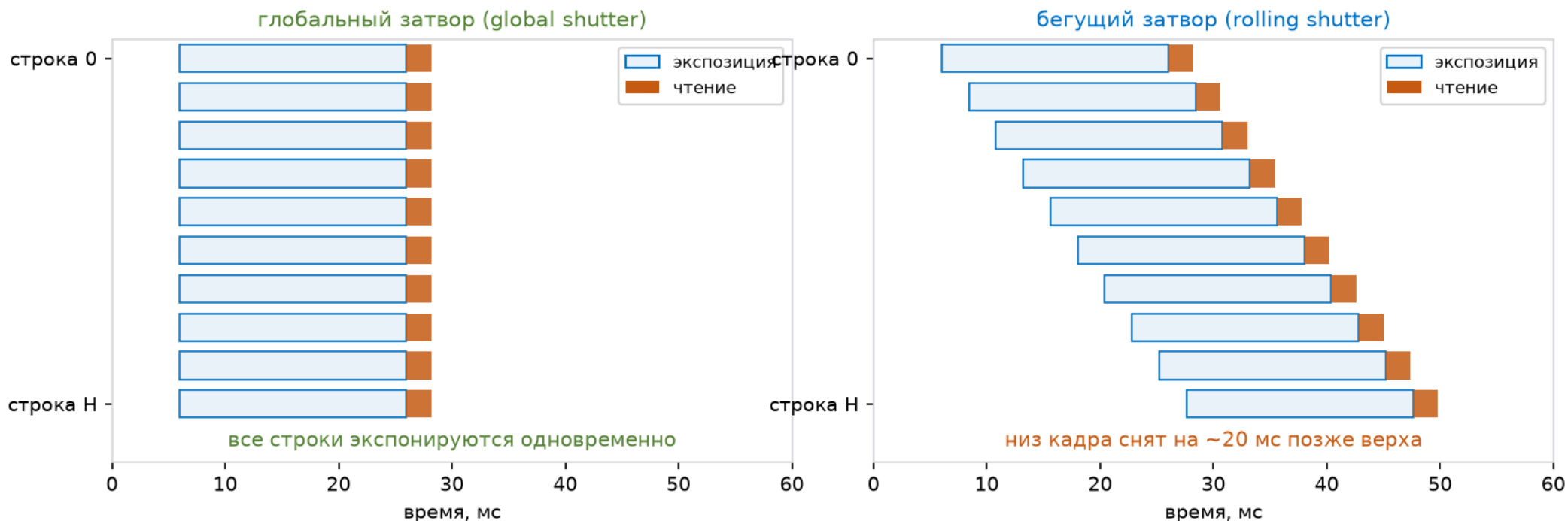
пиксель не «видит цвет» —
он считает электроны,
выбитые фотонами

Пиксель — счётчик: фотоны выбивают электроны с вероятностью 60 % (квантовая эффективность, QE $\approx$ 60 %), колодец вмещает лишь тысячи e^- (full well capacity); выше — только клиппинг (тот самый необратимый пересвет); при чтении добавляется шум $\approx$ 2–3 e^- (шум чтения — это нижний порог сенсора). Три числа — QE, full well, шум чтения — определяют всё, что вы получите с камеры

© собственная иллюстрация

- **CCD**: заряд передаётся по цепочке к одному усилителю — однородно, но медленно и дорого
- **CMOS**: усилитель и АЦП в **каждом пикселе/столбце** — быстро, дёшево, читает окнами
- CMOS выиграл: в телефонах, роботах и камерах 2026 года — практически только он
- Наследие выбора: у большинства CMOS затвор **бегущий** — следующий слайд

Два затвора: глобальный и бегущий



Глобальный затвор (global shutter) экспонирует все строки одновременно; бегущий (rolling shutter) сдвигает окно экспозиции строку за строкой: низ кадра снят на ~20 мс позже верха. Для неподвижной сцены это безразлично. Для движущейся — мир успевает измениться за один кадр. 20 миллисекунд — это много: машина на 60 км/ч проезжает за это время треть метра

© собственная иллюстрация

Rolling shutter гнёт геометрию

глобальный затвор: столбы вертикальны



rolling shutter, панорама: наклон 14°



Панорамирование во время построчного чтения: вертикальные столбы наклонились на 14° . Прямые в мире перестали быть прямыми в кадре — для геометрии блоков III–IV это катастрофа: pinhole-модель предполагает один центр O для всего кадра, а с rolling shutter у каждой строки — свой центр. Отсюда правило: прежде чем чинить алгоритм, проверь затвор

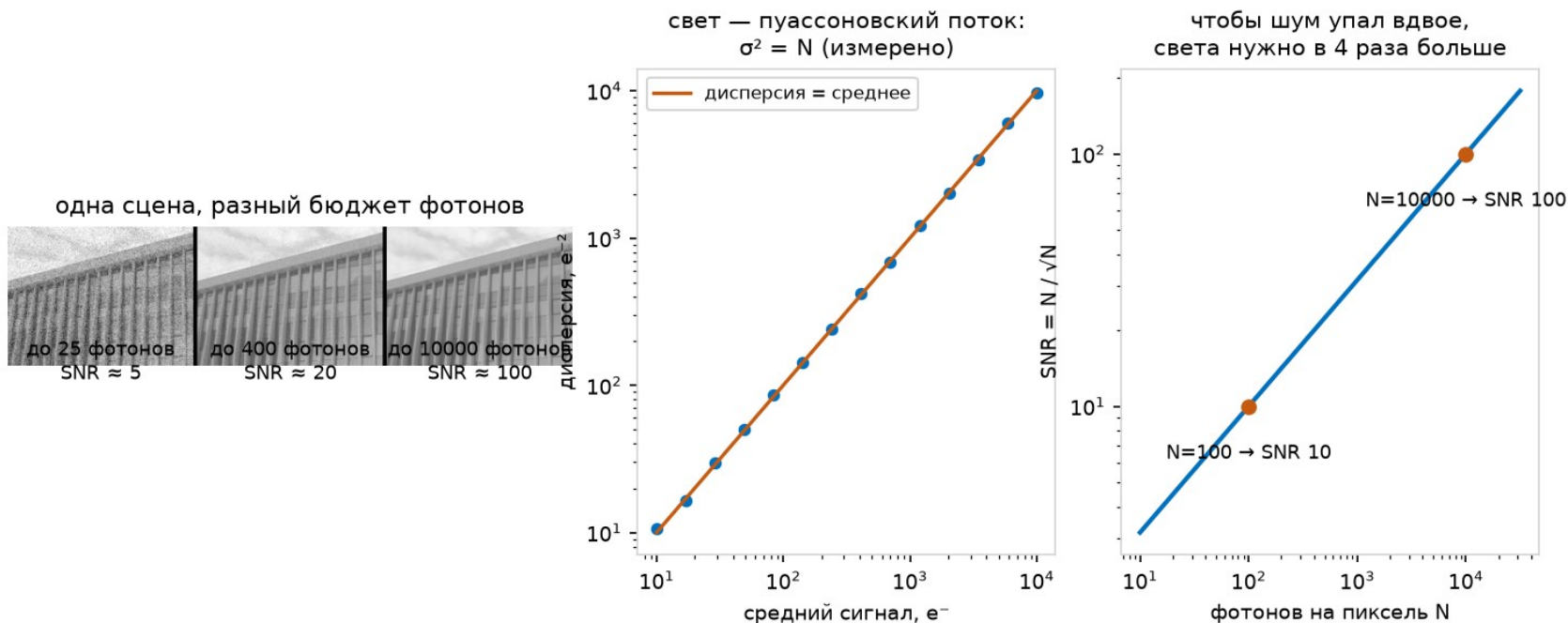
© собственная иллюстрация

- ISO — коэффициент **усиления заряда** перед АЦП; фотонов от него не прибавляется
- Усиливаются поровну сигнал **и весь шум**, накопленный до усилителя
- Шумное ночное фото — вина не ISO, а **малого числа фотонов** при короткой выдержке
- Настоящая цена высокого ISO — **срезанный запас сверху**: $\text{light} \times 2$ при том же колодце

ISO не «добавляет шум» — оно лишь обнажает бедность фотонов. А сам шум — откуда? Есть четыре источника.

- **Фотонный** (shot noise): $\sigma = \sqrt{N}$ — свойство света, фундаментальный, физически неустраним: даже идеальный сенсор шумит, потому что свет приходит квантами и случайно
- **Чтения** (read noise): $\approx 2-3 \text{ e}^-$ на пиксель за кадр — виден только в глубокой тени
- **Тёмновой ток**: тепловые электроны; удваивается каждые $\sim 6-8 \text{ }^\circ\text{C}$ — бич длинных выдержек
- **Структурный** (FPN): пиксели неодинаковы; лечится калибровкой тёмным кадром

Шум измерен: $\sigma^2 = N$



Пуассон подтверждён измерением: дисперсия равна среднему (прямая на графике), $\text{SNR} = \sqrt{N}$. 100 фотонов — SNR 10, богатый свет 10 000 — SNR 100: чтобы шум упал вдвое, света нужно в 4 раза больше

Пиксель 0.7 мкм ловит вчетверо меньше фотонов, чем 1.4 мкм при том же сенсоре — значит, в тени он вчетверо шумнее, и никакие «108 Мп» не спасут

© фото — здание Университета Иннополис; собственная иллюстрация

- DR (Dynamic Range) = full well / шум чтения: $6000 \text{ e}^- / 3 \text{ e}^- = 2000 : 1 \approx 66 \text{ дБ} \approx 11 \text{ стопов}$
- Сцена «солнце + тень» — 17 стопов: **в кадр не влезает** — пересвет или глухие тени
- Отсюда HDR — склейка экспозиций; отсюда же RAW 12–14 бит вместо 8
- Мост к L1: «53 % пикселей упёрлись в 255» — колодцы до краёв

С диапазоном разобрались; переходим к цвету: почему сенсор видит чёрно-белым?

- Кремний считает **все** фотоны видимого диапазона — сам по себе пиксель цвета не знает
- Решение 1976 года: цветной фильтр над **каждым** пикселем — мозаика R, G, G, B
- Зелёных вдвое больше: пик чувствительности глаза; G — носитель яркости
- Цена: каждый пиксель измерил **один** канал из трёх — 2/3 данных о цвете не существует

Что на самом деле видит сенсор

сцена, как её видим мы



что измерил сенсор: одно число на пиксель



фрагмент 8 × 8: RGGB, зелёных — половина

14	62	15	59	62	79	109	66
62	132	62	115	74	57	83	47
11	60	15	59	70	62	106	61
62	130	62	110	90	69	83	47
16	60	19	62	86	82	103	60
59	118	57	101	90	70	77	41
11	37	8	40	97	85	96	55
47	114	56	104	89	70	68	40

RAW — не цветная картинка, а серая шахматка измерений: одно число на пиксель. В четвёрке 8 × 8 видно раскладку RGGB; цветным кадр делает интерполяция — демозаикинг (demosaicing / debayering): алгоритм цифровой обработки изображений, восстанавливающий RGB-изображение из мозаичных данных, полученных с матрицы сенсора

© фото — здание Университета Иннополис; собственная иллюстрация

Демозаикинг и его артефакты

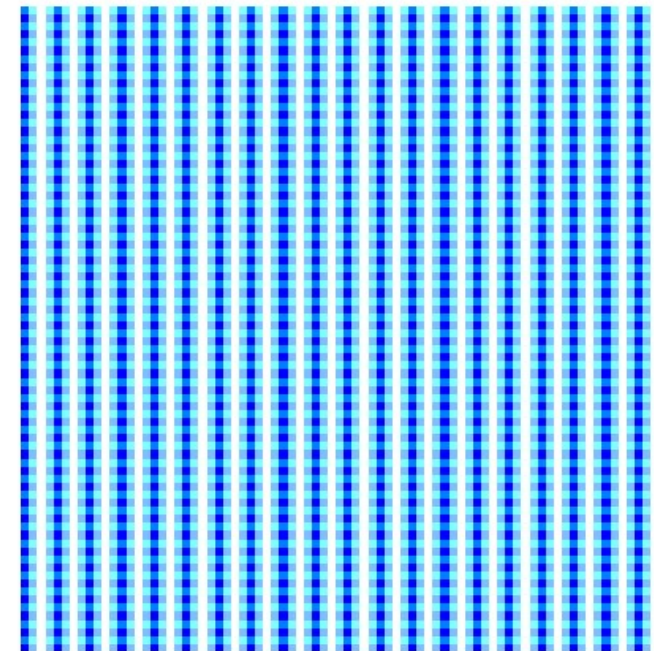
оригинал (фрагмент с решётками окон)



после мозаики и демозаика: PSNR 29.0 дБ



линии 1 px: ложный цвет
на 75 % пикселей



Демозаикинг интерполирует $2/3$ значений от соседей: на нашем кадре PSNR 29.0 дБ — заметно глазу. Где сосед врёт — на структурах в 1–2 px рождается ложный цвет: на мишени из чёрных линий он занял 75 % пикселей. Отсюда правило: тонким структурам нельзя верить в цвете — каёмки на решётках и текстах — фантомы демозаика

© фото — здание Университета Иннополис; собственная иллюстрация

ISP: конвейер за один взгляд

ISP: каждая стадия делает кадр приятнее для глаза — и дальше от измерения



ISP (image signal processor) внутри камеры: black level, демозаикинг, баланс белого, гамма, резкость, JPEG. Каждая стадия делает кадр приятнее для глаза — и дальше от измерения. Важно: порядок стадий фиксирован, и всё после демозаика работает с уже выдуманными значениями. Пройдём по стадиям и посмотрим, что каждая даёт глазу и что отнимает у измерения. Первая — баланс белого: процесс цветокоррекции, который делает цвета на фотографии или видео естественными, компенсируя влияние цвета освещения

© фото — здание Университета Иннополис; собственная иллюстрация

Баланс белого: серый мир при автонастройке

сцена при белом свете

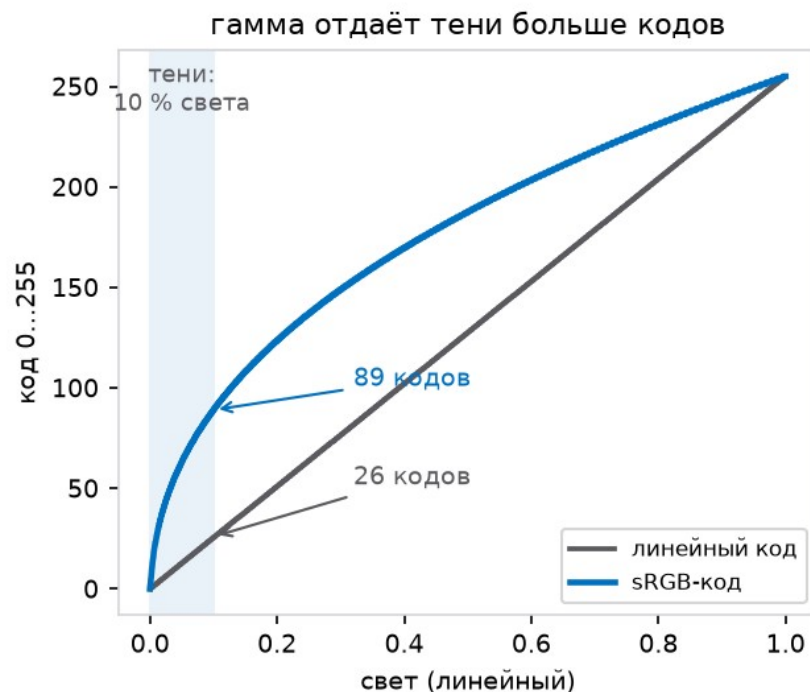


тёплая лампа (В × 0.5): сенсор честен «серый мир» дал В лишь ×1.17: сцена не серая

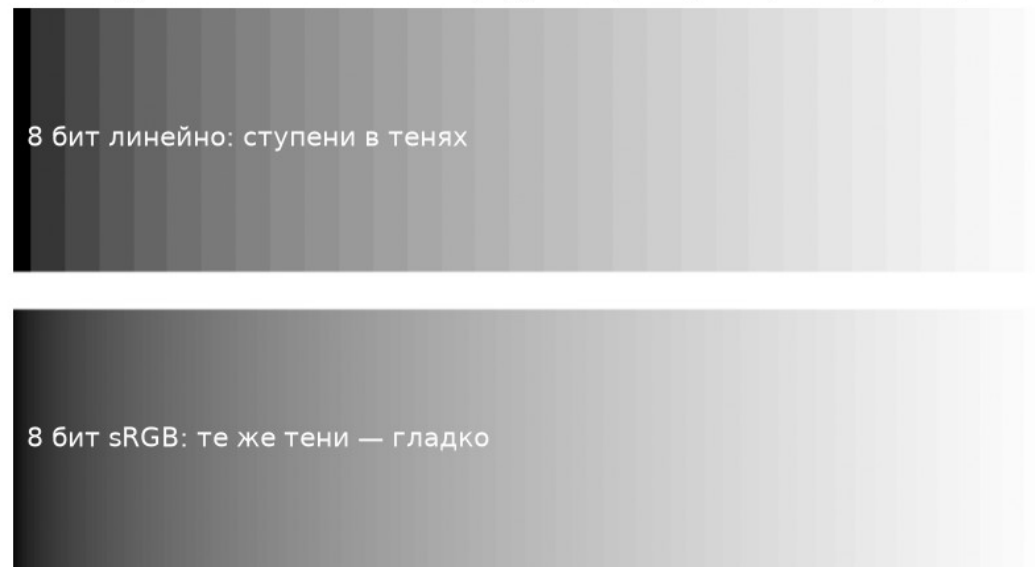


Исходная гипотеза: алгоритм предполагает, что если усреднить все пиксели изображения, то в сумме должен получиться нейтральный серый цвет. Если среднее смещено (например, в желтизну), алгоритм сдвигает цвета в противоположную сторону, чтобы вернуть баланс. Например, кадр был снят при искусственном тёплом освещении, из-за чего сенсор зафиксировал сильный жёлтый оттенок. Однако на фотографии огромную часть кадра занимает синее небо. Результат: синее небо искажает общую статистику кадра. Из-за обилия синего цвета алгоритм «подумал», что сцена и так достаточно холодная. Вместо того чтобы компенсировать желтизну лампы и поднять синий канал (В) в 2 раза ($\times 2$), он увеличил его всего в 1.17 раза ($\times 1.17$)

© фото — здание Университета Иннополис; собственная иллюстрация



один и тот же тёмный градиент (контраст растянут $\times 8$)



Гамма-коррекция позволяет эффективно распределить биты информации, сохраняя плавность деталей в тенях (тёмных участках). Сенсор линеен: вдвое больше фотонов — вдвое заряд. Но линейные 8 бит отдают нижним 10 % света лишь 26 кодов — в тенях ступени. Гамма-кривая sRGB выдаёт тем же теням 89 кодов: та же разрядность — гладкие тени. Вот что прячет гамму в uint8 — ISP; вот почему «код 128 — не половина света»

© собственная иллюстрация

- Повышение резкости (unsharp mask): создаёт вокруг контуров искусственные ободки (ореолы). Это путает алгоритмы поиска границ и распознавания объектов
- Ореолы — **края, которых нет в сцене**: их найдёт Canny (L7) и сматчит дескриптор (L9)
- Шумоподавление: размывает пиксели, чтобы убрать зернистость. Вместе с шумом оно стирает мелкие **текстуры**, которые необходимы компьютеру для анализа
- Правило: для задач компьютерного зрения **отключайте** обе функции (снимайте в RAW), так как «красивая» для человека картинка вредит точности вычислений

Что же остаётся, если отказаться от всех фиц ISP? RAW!

Критерий	RAW («сырые» данные)	JPEG (готовая картинка)
Глубина цвета	12–14 бит: огромный запас информации в каждом пикселе	8 бит: цвета уже сжаты гамма-кривой, информации меньше
Цвет и структура	Сырая мозаика: можно применить свой, более точный алгоритм демозаика	Цвета уже зафиксированы: процессор камеры сам восстановил цвета, исправить ошибки нельзя
Баланс белого	Настраивается потом: цвет освещения можно без потерь изменить на компьютере	Вшит намертво: камера уже применила свои настройки баланса белого
Диапазон яркости	Максимальный: сохраняются все детали как в глубоких тенях, так и в ярких бликах	Ограниченный: сильно контрастные детали могут быть безвозвратно потеряны
Размер файла	Большой: из-за отсутствия сжатия файлы весят много	В 10–20 раз меньше: занимает мало места, открывается на любом устройстве
Скорость работы	Требует времени: файлы нужно долго декодировать и обрабатывать	Мгновенно: обрабатывается встроенным чипом камеры на лету

Операция	Зачем глазу	Что теряет измерение
Black level (вычитание порогового шума)	Делает чёрное — чёрным	почти безвредно: вычитание
Демозаик (восстанавливает полноценный цвет RGB в каждой точке)	Восстанавливает цвет в каждом пикселе	2/3 цветов — догадка (интерполяция)
Баланс белого (подстраивает цвета под освещение, чтобы белые объекты оставались белыми)	Цвета «как при белом»	связь с реальным физическим спектром света (спектральный состав сцены восстановить больше невозможно)
Гамма-коррекция (сжимает данные в 8 бит так, чтобы тени оставались плавными и приятными для зрения)	Гладкие тени в 8 бит	уничтожается линейность: коды $\neq$ свет
Резкость, шумоподавление (делает контуры объектов чёткими, а саму картинку чистой от «зерна»)	«Чёткость», чистота	стирает мелкие текстуры, а повышение резкости создаёт ложные края (ореолы), сбивающие с толку детекторы контуров
JPEG (сжатие)	Лёгкий файл, который быстро открывается на любом телефоне или компьютере	изображение разбивается на блоки 8×8 пикселей, внутри которых усредняются цвета и стираются критически важные для анализа микродетали

Робототехника

- выбор камеры: f_x под дистанцию, затвор под движение, экспозиция руками
- монокулярная дальность и калибровка — L11–L12 стоят на модели K
- шум $\sqrt{N}$ решает, полетит ли ночной дрон

МОИИ

- аугментации = имитация сенсора: шум Пуассона, гамма, дисторсия
- датасет уже прошёл чужой ISP — его артефакты выучит модель
- RAW-датасеты и denoising: работают в **линейном** сигнале

- Пиксель **считает фотоны**: $\sqrt{N}$, full well и шум чтения задают всё — от ISO до HDR
- Цвет — выдумка демозаика поверх мозаики Байера; затвор бывает бегущим
- ISP делает кадр для глаза; измерению остаётся RAW — или трезвое знание потерь

Демо 2.1 — мозаика и демозаик

python

```
1 import cv2
2 from cvcourse import io as cio, metrics
3 img = cio.imread("demo/iu-building.jpg")
4 mosaic = np.zeros(img.shape[:2], np.uint8)    # RGGG: R в (0,0)
5 mosaic[0::2, 0::2] = img[0::2, 0::2, 2]      # R
6 mosaic[0::2, 1::2] = img[0::2, 1::2, 1]      # G
7 mosaic[1::2, 0::2] = img[1::2, 0::2, 1]      # G
8 mosaic[1::2, 1::2] = img[1::2, 1::2, 0]      # B
9 back = cv2.cvtColor(mosaic, cv2.COLOR_BayerBG2BGR)
10 print("PSNR после демозаика: %.1f дБ" % metrics.psnr(img, back))
```

PSNR после демозаика: 29.0 дБ

Вызов

Что делает

`mosaic[0::2, 0::2] = img[..., 2]`

срезы с шагом 2 раскладывают каналы в шахматку RGGG — мозаика в 4 строки

`cv2.cvtColor(m, COLOR_BayerBG2BGR)`

демозаик; код «BG» называет цвета **второй** строки шаблона, не угла (0,0)

Демо 2.2 — ложный цвет на 1 px

python

```
1 t = np.full((200, 320), 255, np.uint8) # серая мишень: R=G=B
2 t[:, ::4] = 0                          # чёрные линии в 1 px
3 back = cv2.cvtColor(t, cv2.COLOR_BayerBG2BGR).astype(int)
4 false = (np.abs(back[..., 2] - back[..., 0]) > 32).mean()
5 print("ложный цвет: %.0f %% пикселей" % (100 * false))
```

ложный цвет: 75 % пикселей

Вызов

`t[:, ::4] = 0`

`np.abs(R - B) > 32`

Что делает

вертикальная линия толщиной 1 px каждые 4 столбца — худший случай для демозаика

пиксель «цветной», если каналы разошлись больше чем на 32, — а сцена-то серая

Демо 2.3 — шум и стек кадров

```
1 rng = np.random.default_rng(0)
2 N = 400                                # фотонов на пиксель
3 stack = rng.poisson(N, (16, 200, 200)).astype(np.float32)
4 print("один кадр:   $\sigma = %.1f$  ( $\sqrt{N} = %.1f$ )"
5       % (stack[0].std(), N ** 0.5))
6 print("среднее 16:  $\sigma = %.1f$  — в 4 раза меньше"
7       % stack.mean(0).std())
```

python

один кадр: $\sigma = 20.1$ ($\sqrt{N} = 20.0$) · среднее 16
кадров: $\sigma = 5.0$ — в 4 раза меньше

Вызов

`rng.poisson(N, (16, h, w))`

Что делает

16 кадров одной сцены с честным фотонным шумом; зерно фиксировано

`stack.mean(0)`

усреднение по оси кадров: сигнал тот же, σ падает в $\sqrt{16} = 4$ раза

Демо 2.4 — rolling shutter пуками

```
1 frame = np.full((400, 600), 235, np.uint8)
2 for x0 in (120, 300, 480):
3     frame[:, x0:x0 + 26] = 40          # три вертикальных столба
4 rs = np.stack([np.roll(frame[y], y // 4) for y in range(400)])
5 print("панорама при чтении: наклон %.0f°"
6       % np.degrees(np.arctan(0.25)))
```

python

панорама при чтении: наклон 14°

Вызов

`np.roll(frame[y], y // 4)`

`np.arctan(0.25)`

Что делает

строка y снята позже и видит сцену сдвинутой на $y/4$ px — имитация бегущего затвора

наклон вертикали: арктангенс сдвига на строку — геометрия артефакта предсказуема

Фрагмент «оценить свет и шум»

- `img = cv2.imread("frame.jpg")`
- `light = img.mean()`
- `noise = img[:60, :60].std()`

Три ошибки

- кадр после ISP: резкость и JPEG уже исказили и края, и статистику
- `mean()` по sRGB-кодам — не свет: гамма; среднее кодов $\neq$ средняя яркость сцены
- `std()` угла кадра мешает шум с **текстурой**; шум меряют по стеку кадров

ПОДВОДНЫЕ КАМНИ

- **«Больше мегапикселей — лучше»:** мелкий пиксель ловит меньше фотонов $\rightarrow \text{SNR} \propto \sqrt{N}$ падает
- **«ISO шумит»:** усиление множит сигнал и шум одинаково; шумит малое число фотонов
- **Автоэкспозиция и автофокус при измерениях:** кадры серии сняты «разными камерами»

ПОДВОДНЫЕ КАМНИ

- **Тонкой структуре нельзя верить цвет:** каёмки на решётках и текстах — фантомы демозаика
- **Rolling shutter в движении:** «прямые» в кадре гнутся — прежде чем чинить алгоритм, проверь затвор
- **Ореолы резкости — не края сцены:** Sanny и дескрипторы найдут то, что дорисовал ISP

KEY TAKEAWAYS

- Камера — угломер: $u = f_x \cdot X/Z + c_x$; при 60° и 1920 px $f_x = 1663$, и пешеход 1.7 м на 20 м — это 141 px
- Свет — бюджет: выдержка, диафрагма, усиление делят его на троих; цены — смаз, малая ГРИП, шум
- Пиксель считает фотоны: $\sigma = \sqrt{N}$ — вдвое чище значит вчетверо больше света; стек из 16 кадров режет шум в 4 раза
- Кадр после ISP сделан для глаза, **не для измерений**: 2/3 цветов выдуманы (на 1-px линиях ложный цвет на 75 % пикселей), гамма спрятала линейность, резкость дорисовала края
- Дальше камера в курсе — это числа f_x , f_y , c_x , c_y и $k_1 \dots$: в L11 измерим их по шахматной доске

САМОПРОВЕРКА

- Камера 90° и 1280 px: чему равен f_x , и сколько пикселей займёт дверь высотой 2 м на 8 м?
- Почему усреднение 16 кадров снижает шум в 4 раза, а не в 16? Каким станет σ при 64 кадрах?
- В кадре 8×8 с мозаикой RGGB — сколько **измеренных** зелёных значений и сколько выдумает демозаик?
- Дрон летит вдоль забора, съёмка на rolling shutter: что произойдёт с вертикальными досками и почему?

Разбор — в начале L3, на слайде «Опоры»

ДАЛЬШЕ

- **Занятие 2 «От RAW к RGB»:** мозаика Байера из своего фото, демозаик руками + PSNR против cvtColor, шум камеры по стеку кадров; ★ rolling shutter
- **Мост:** заготовка начинается ровно на демо 2.1 и 2.3 этой лекции
- **ДЗ 1 «Фотолаборатория»** — выдача на W3, дедлайн W6

Ноутбук и данные — в репозитории за сутки до пары

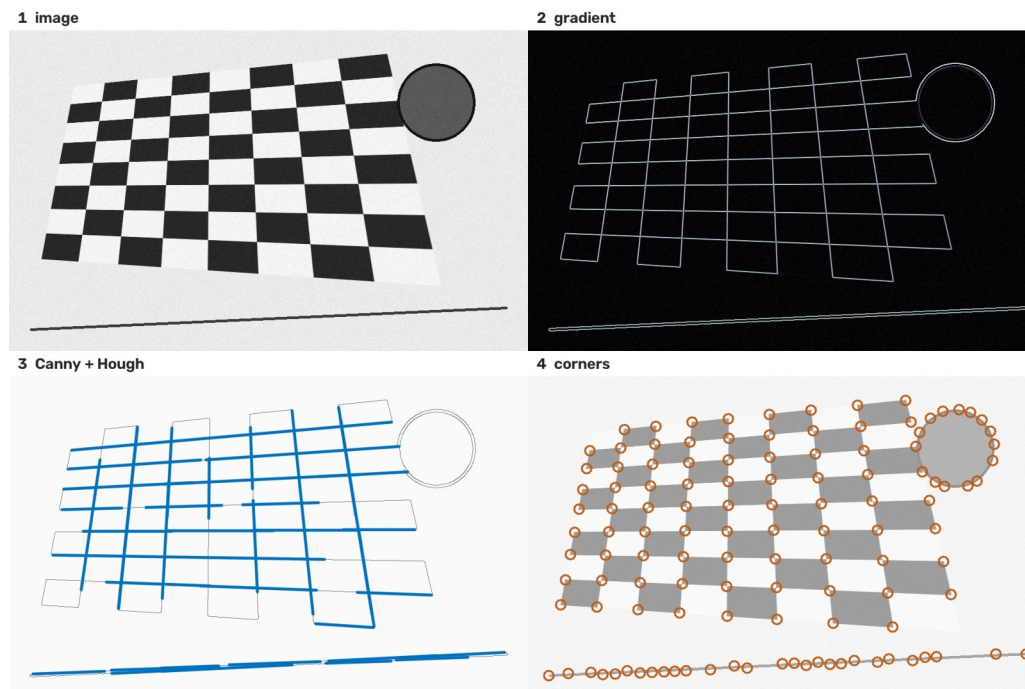
REFERENCES

- Szeliski R. *Computer Vision: Algorithms and Applications*, 2nd ed., 2022 — гл. 2.2–2.3: формирование изображения, линзы, сенсор. К следующей паре: §2.3.2 (цвет)
- Nayar S. *First Principles of Computer Vision*, Columbia, YouTube — плейлисты Imaging и Sensing: лучшие анимации по сегодняшним темам
- EMVA Standard 1288 — как индустрия измеряет QE, шум и динамический диапазон сенсора
- Poynton C. *Digital Video and HD*, 2nd ed., 2012 — гамма без мифов
- OpenCV docs 4.x: `cvtColor` (Bayer → BGR), модуль `calib3d` — модель дисторсии
- Bayer B. *Color imaging array*, патент US 3 971 065, 1976 — три страницы первоисточника

ТОЛКОВЫЙ СЛОВАРЬ

Термин	Что это значит
камера-обскура (camera obscura)	тёмная камера с отверстием: изображение из одной геометрии прямых лучей
фокусное расстояние (focal length)	масштаб проекции; в пикселях $f_x = W / (2 \cdot \tan(\text{FOV}/2))$
главная точка (principal point)	пиксель (c_x, c_y) , в который падает оптическая ось; $\approx$ центр кадра
диафрагменное число (f-number)	$N = f/D$; на шаг ряда больше — вдвое меньше света, ГРИП глубже
глубина резкости (depth of field)	зона дистанций, где кружок нерезкости меньше допуска
кружок нерезкости (circle of confusion)	пятно, в которое расплывается точка не в фокусе
дисторсия (distortion)	искривление проекции объективом: $r' = r \cdot (1 + k_1 r^2 + k_2 r^4)$
квантовая эффективность (quantum efficiency)	доля фотонов, ставших электронами: у современных CMOS 50–80 %
полная ёмкость (full well capacity)	предел электронов в пикселе; достигли — клиппинг, пересвет необратим
фотонный шум (shot noise)	случайность прихода фотонов: $\sigma = \sqrt{N}$; физически неустраним
динамический диапазон (dynamic range)	отношение full well к шуму чтения — в стопах или децибелах
мозаика Байера (Bayer pattern)	RGGB-фильтры над пикселями: одно цветовое измерение на пиксель
демозаик (demosaiicing)	интерполяция двух недостающих каналов; 2/3 значений цвета — оценка
бегущий затвор (rolling shutter)	построчное чтение: строки кадра сняты в разные моменты времени
ISP (image signal processor)	процессор камеры: конвейер от RAW до JPEG, оптимизирует под глаз

Вопросы?



Dr. Ilya Afanasyev · i.afanasyev@innopolis.ru

Демо-код лекции: [lectures/L02-optics-sensor/demo/](#)

Занятие 2 «От RAW к RGB» — принесите своё фото